Trabalho Estrutura de Dados

Ygor de Souza Oliveira - ADS

1- Uma `struct` em C é uma estrutura que permite **agrupar diferentes tipos de dados dentro de uma única variável**.

```
struct Aluno {
    char nome[50];
    int idade;
    float nota;
};
```

2- Serve para alocar memória dinamicamente durante a execução de um programa em C.

```
int *numeros;
numeros = malloc(5 * sizeof(int));
```

3-

```
#include <stdio.h>

#define TAMANHO 5

int pilha[TAMANHO];
int topo = -1;

void push(int valor) {
    if (topo == TAMANHO - 1) {
        printf("Pilha cheia!\n");
    } else {
        topo++;
        pilha[topo] = valor;
        printf("Elemento %d inserido.\n", valor);
    }
}

void pop() {
    if (topo == -1) {
        printf("Pilha vazia!\n");
```

```
    } else {
        printf("Elemento %d removido.\n", pilha[topo]);
        topo--;
    }
}

void mostrar() {
    if (topo == -1) {
        printf("Pilha vazia!\n");
    } else {
        printf("Elementos da pilha:\n");

        for (int i = topo; i >= 0; i--) {
            printf("%d\n", pilha[i]);
        }
    }
}

int main() {
    int opcao, valor;

    do {
        printf("\n--- PILHA ---\n");
        printf("1 - Inserir elemento\n");
        printf("2 - Remover elemento\n");
        printf("3 - Mostrar pilha\n");
        printf("0 - Sair\n");
        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);

        switch (opcao) {
            case 1:
                printf("Digite o valor: ");
                scanf("%d", &valor);
                push(valor);
                break;

            case 2:
                pop();
                break;

            case 3:
                mostrar();
                break;
```

```
            case 0:
                printf("Programa encerrado.\n");
                break;

            default:
                printf("Opcao invalida!\n");
        }

    } while (opcao != 0);

    return 0;
```

4- }**Resposta:**

```
#include <stdio.h>

#define TAMANHO 5

int fila[TAMANHO];
int inicio = 0;
int fim = 0;

void inserir(int valor) {
    if (fim == TAMANHO) {
        printf("Fila cheia!\n");
    } else {
        fila[fim] = valor;
        fim++;
        printf("Elemento %d inserido.\n", valor);
    }
}

void remover() {
    if (inicio == fim) {
        printf("Fila vazia!\n");
    } else {
        printf("Elemento %d removido.\n", fila[inicio]);
        inicio++;
    }
}

void mostrar() {
    if (inicio == fim) {
```

```
        printf("Fila vazia!\n");
    } else {
        printf("Elementos da fila:\n");

        for (int i = inicio; i < fim; i++) {
            printf("%d ", fila[i]);
        }

        printf("\n");
    }
}

int main() {
    int opcao, valor;

    do {
        printf("\n--- FILA ---\n");
        printf("1 - Inserir elemento\n");
        printf("2 - Remover elemento\n");
        printf("3 - Mostrar fila\n");
        printf("0 - Sair\n");
        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);

        switch (opcao) {
            case 1:
                printf("Digite o valor: ");
                scanf("%d", &valor);
                inserir(valor);
                break;

            case 2:
                remover();
                break;

            case 3:
                mostrar();
                break;

            case 0:
                printf("Programa encerrado.\n");
                break;

            default:
```

```
            printf("Opcao invalida!\n");
        }

    } while (opcao != 0);

    return 0;
}
```

5- Resposta:

```
struct No {

    int valor;

    struct No *proximo;

};
```

O ponteiro `proximo` serve para armazenar o endereço do próximo nó da lista encadeada. Dessa forma, um nó consegue apontar para o próximo nó, formando a sequência da lista.

No último nó, o ponteiro `proximo` recebe `NULL`, indicando que não existem mais elementos.

6- Resposta:

```
#include <stdio.h>

#include <stdlib.h>


struct No {

    int valor;

    struct No *proximo;

};
```

```
int main() {
    struct No *no1;
    struct No *no2;
    struct No *no3;


    no1 = malloc(sizeof(struct No));
    no2 = malloc(sizeof(struct No));
    no3 = malloc(sizeof(struct No));


    no1->valor = 10;
    no2->valor = 20;
    no3->valor = 30;


    no1->proximo = no2;
    no2->proximo = no3;
    no3->proximo = NULL;


    printf("%d -> ", no1->valor);
    printf("%d -> ", no2->valor);
    printf("%d -> NULL\n", no3->valor);
```

```
    free(no1);

    free(no2);

    free(no3);


    return 0;

}
```

Saída esperada:

10 -> 20 -> 30 -> NULL

7- Resposta:

- **Vetor: é uma estrutura que armazena vários elementos do mesmo tipo em posições consecutivas de memória. Cada elemento pode ser acessado por meio de um índice.**
- **Pilha: é uma estrutura em que o último elemento que entra é o primeiro que sai. É conhecida como LIFO (*Last In, First Out*).**
- **Fila: é uma estrutura em que o primeiro elemento que entra é o primeiro que sai. É conhecida como FIFO (*First In, First Out*).**
- **Lista Encadeada: é uma estrutura formada por nós. Cada nó armazena um valor e um ponteiro que indica onde está o próximo nó da lista.**

8- `push(10)` **→ 10**

1. `push(20)` **→ 10, 20**
2. `push(30)` **→ 10, 20, 30**
3. `pop()` **→ remove 30 → 10, 20**
4. `push(40)` **→ 10, 20, 40**

5. `pop()` → remove 40 → 10, 20

Resposta:

O conteúdo final da pilha será:

10 → 20`inserir(10)` **→ 10**

9-

1. `inserir(20)` → 10, 20
2. `inserir(30)` → 10, 20, 30
3. `remover()` → remove o 10 → 20, 30
4. `inserir(40)` → 20, 30, 40
5. `remover()` → remove o 20 → 30, 40

Questão 09 — Conteúdo final da fila

Resposta:

O conteúdo final da fila será:

30 → 40

O elemento que está na frente da fila é o 30.

O elemento que está no topo é o 20.

10- O código possui 3 erros principais de sintaxe e lógica , além de não exibir as inclusões de biblioteca necessárias.